Useful resources:

- Multer Official Docs: https://github.com/expressjs/multer
- Streaming Files: https://medium.freecodecamp.org/node-js-streams-everything-you-need-to-know-c9141306be93
- Generating PDFs with PDFKit: http://pdfkit.org/docs/getting_started.html

---

```
const fileStorage = multer.diskStorage({
  // 1. الوجهة: بنحدد الفولدر اللي الصور هتروح فيه
  destination: (req, file, cb) => {
    // cb (callback) بتاخد (error, destination)
    // null معناها مفيش error، و "images" هو اسم الفولدر
    cb(null, "images");
  },
  // 2. التسمية: بنحدد الاسم اللي هيظهر به الملف في الهارد
  filename: (req, file, cb) => {
    // ملاحظة: لازم تتكتب filename (الـ n سمول) مش fileName
    // بنضيف التاريخ (Date) بنفس لاسم الملف الأصلي عشان لو يوزرين رفعوا صورتين بنفس
الاسم ميمسحوش بعض
    cb(null, new Date().toISOString().replace(/:/g, '-') + "_" +
file.originalname);
  },
});
```

```
const fileFilter = (req, file, cb) => {
  // هو الهوية الحقيقية للملف اللي بيبعتها المتصفح (mimetype) الميم تايب
  // ملحوظة: الـ mimetype بيبقى "image/png" مش "image.png"
  if (
    file.mimetype === "image/png" ||
    file.mimetype === "image/jpg" ||
    file.mimetype === "image/jpeg"
  ) {
    cb(null, true); // اقبل الملف ده يا ريس
  } else {
    cb(null, false); // ارفض الملف ده وماتسيفوش
  }
};
```

```
// بنحقن الإعدادات في Multer
app.use(
  multer({
    storage: fileStorage, // استخدم المحرك اللي فصلناه فوق
    fileFilter: fileFilter // استخدم الفلتر اللي حددناه
  }).single("image") // بتاعه name اللي الـ input استقبل ملف واحد من الـ
"image"
);
```

إنت دلوقتي سيفت الصورة جوه فولدر `images` على السيرفر، بس لو حاولت تفتح الرابط بتاعها في المتصفح (مثلاً `localhost:3000/images/my-pic.jpg`) هيطلعلك **Not Found 404**.

ليه؟ لأن الـ Express بطبيعته "مقفل"؛ أي فولدر أو ملف عندك في المشروع هو ملف "سري" المتصفح ميعرفش يشوفه، إلا لو إنت "فتحته" للجمهور. عملية **Serving the images** هي إنك بتخلي الفولدر ده **Static** (متاح للكل).

---

1. المفهوم: الـ Static Routing

في العادي، أي Route بيبدأ بـ `app.get` أو `app.post` . لكن الصور مش محتاجة Logic، هي محتاجة السيرفر يبعت الملف زي ما هو للهارد ديسك بتاع اليوزر. ده اللي بنسميه **Serving Files Statically**.

---

2. الكود السحري: `express.static`

عشان تفتح الفولدر ده للناس، بتستخدم الميدل وير ده في `app.js` :

JavaScript

```
const path = require('path');

// بتقول للـ Express: أي حد يطلب حاجة بتبدأ بـ /images
// في المشروع بتاعي روح دور عليها جوه الفولدر اللي اسمه images
app.use('/images', express.static(path.join(__dirname, 'images')));
```

شرح السطر ده:

1. `app.use('/images', ...)` : ده اسمه "Virtual Path". يعني لما اليوزر يكتب في المتصفح `/images/123.jpg` .. الـ Express هيفهم إنه رايح للفولدر ده.
2. `express.static` : ده الميدل وير اللي بيقرأ الملف من الهارد ديسك ويبعته للمتصفح أوتوماتيك.

3. `path.join(__dirname, 'images')` : ده بيحدد المسار الحقيقي للفولدر على جهازك (Absolute Path).

---

3. إزاي بنستخدمها في الـ EJS؟

لما تيجي تعرض الصورة في صفحة الـ Shop مثلاً، إنت مخزن في الداتابيز الـ Path اللي هو `images/2026-04-25_pic.jpg` .

في الـ EJS هتكتب:

HTML

```
<img src="/<%= product.imageUrl %>" alt="<%= product.title %>">
```

المتصفح هيروح يطلب: `http://localhost:3000/images/2026-04-25_pic.jpg` . بما إننا عملنا **Serving** للفولدر ده، السيرفر هيرد بالصورة فوراً.

---

1. الـ `fs.createReadStream` (فتح الماسورة)

بدل ما نفتح الملف كله ونرميه في الرام (Buffer)، إحنا بنفتح **Read Stream**.

- **المفهوم:** الـ Stream ده عبارة عن "تتابع من البيانات" (Sequence of data) متاح بمرور الوقت. نود بتبدأ تقرأ الملف حتة حتة (Chunks)، وكل حتة في العادي حجمها **64KB**.
- **ليه ده صياعة؟** لو الملف حجمه 1GB، السيرفر بتاعك هيفضل مستهلك **64KB** بس من الرام وهو بيبعته. لو استخدمت `readFile` العادية، السيرفر هيحاول يحجز 1GB في الرام، ولو فيه 5 يوزرز مع بعض، السيرفر هيقولك "باي باي" ويحصل **Memory Overflow**.

---

2. الـ `res.setHeader` (توجيهات السفر)

المتصفح لما بيستلم داتا، بيبقى تايه مش عارف دي إيه. الـ Headers دي هي "الكتالوج" اللي بيفهمه.

- `Content-Type: application/pdf` :
  - بتقول للمتصفح: "يا ريس، اللي جاي ده مش نص ولا صورة، ده ملف PDF سليم". ده بيخلي المتصفح يشغل الـ **PDF Viewer** الداخلي بتاعه.
- `Content-Disposition` :
  - `inline` : لو عايز اليوزر يحمله فوراً كملف بتغيرها لـ `attachment`). جديدة Tab دي معناها "اعرضه قدامي". المتصفح هيفتحه في

- لو مبعتوش، المتصفح هيسميه اسم "Save". ده الاسم اللي هيظهر لما اليوزر يقرر يدوس : `filename="..."` عشوائي أو باسم الـ Route.

---

3. الـ `file.pipe(res)` (السحر كله هنا)

الـ `pipe` دي هي "الوصلة" اللي بتربط ماسورة القراية بماسورة الرد.

- **الكواليس:** الـ `res` (الـ Response) في نود هو أصلاً **Writable Stream**. لما بتقول `file.pipe(res)`، إنت بتقول لنود: "كل حتة (Chunk) تقراها من الملف، ابعتها فوراً في الـ Response".
- **الـ Backpressure (ضغط البيانات):** دي أهم حتة تقنية. الـ `pipe` ذكية جداً؛ لو المتصفح بطيء مش قادر يستلم الداتا بسرعة، الـ `pipe` بتهدي سرعة القراية من الهارد ديسك عشان الـ Buffer ميتمليش ع الفاضي. لما المتصفح يفضى، ترجع تقرأ تاني. دي عملية موازنة تلقائية مستحيل تعملها يدوي بـ `readFile`.